SoilQ

Quantum-Assisted Maize Agronomy & Hybrid QML Benchmark

Qiskit Fall Fest Nairobi 2026 — Challenge 08: Quantum AI Classifier

Participant	Edwin Mwiti
Email	eduedwyn5@gmail.com
Institution	Kirinyaga University
Repository	https://github.com/Edwin420s/SoilQ
Implementation	Python · Qiskit · scikit-learn · FastAPI · Streamlit
Submission status	Research prototype; dataset-specific results pending verification

Submission note: This report documents the implementation and the reproducible experiment workflow. It deliberately does not present unverified dataset counts or model scores as completed results.

1. Abstract

SoilQ is a research prototype exploring whether a small Qiskit variational quantum classifier can classify a binary agronomic outcome and how it compares with classical machine-learning baselines. The intended application context is Kenyan maize agronomy, with a focus on transparent, evidence-based analysis rather than automatic irrigation control.

The codebase includes a dataset-audit utility, a model-training pipeline, three classical baselines, a Qiskit Statevector-based variational classifier, JSON experiment reports, a FastAPI interface, a Streamlit dashboard, tests, and reproducibility documentation. The Kenyan maize dataset is not bundled in the codebase; the actual files must be downloaded and audited before selecting the target and feature columns.

2. Problem formulation and data integrity

The initial data source under consideration is the Kenyan maize field experiment dataset hosted on Mendeley Data: https://data.mendeley.com/datasets/8bx88zpvdp/2. Its published description refers to soil moisture at crop stages, rainfall, plant measurements and yield-related outcomes across Kenyan field experiments. The exact downloadable schema, sampling structure, units, missingness and row count must be confirmed from the files themselves.

Target discipline. SoilQ does not automatically create an “irrigation urgency” label from yield or other variables. A valid binary target must be directly supplied by the source or be a clearly justified proxy with its definition, units and limitations documented. If the dataset is too sparse or does not support the proposed target, the research question must be revised rather than forcing a label.

Leakage checks. Rows may share a site, plot, treatment, season or repeated-measurement group. A random stratified split is only a baseline and may overestimate generalization when related observations appear in both train and test sets. Group-aware or chronological validation should be used when appropriate.

3. Software architecture

Component	Purpose
Dataset audit	Inventories supported table files; reports columns, dtypes, missingness, unique counts, sample values, candidate co
Classical benchmarks	Logistic Regression, RBF-kernel SVM, and Random Forest with preprocessing fit on training data only.
Quantum classifier	Small parameterized circuit with RY feature encoding, trainable RY/RZ rotations, CNOT entanglement, and an expe
Experiment report	Records input hash, selected target/features, class mapping, split settings, metrics, confusion matrix, fit time and mo
API and dashboard	FastAPI audit/train endpoints and a Streamlit interface for audits and report inspection.
Quality tooling	Pytest smoke tests, pyproject configuration, Docker files, CLI scripts and research documentation.

4. Qiskit implementation

The implementation uses Qiskit’s local Statevector simulator. For each example, numeric features are scaled to [-1, 1] and encoded as RY(πx■) rotations. Each variational layer applies trainable RY and RZ rotations followed by CNOT entanglement between neighboring qubits; for more than two qubits, a closing CNOT forms a ring. The model uses the expectation value of Pauli-Z on qubit 0 as its score.

The trainable parameters are optimized with COBYLA against a regularized logistic-style loss. The current implementation uses up to six features and a small number of circuit repetitions to keep the simulation manageable.

```
# Core circuit pattern (simplified)
for q in range(n_qubits):
    qc.ry(pi * x[q], q)       # data encoding
for layer in range(reps):
    for q in range(n_qubits):
        qc.ry(theta[k], q)
        qc.rz(theta[k + 1], q)
    for q in range(n_qubits - 1):
        qc.cx(q, q + 1)
```

```
    if n_qubits > 2:
        qc.cx(n_qubits - 1, 0)
```

This is a classical simulation of a quantum circuit, not execution on quantum hardware. The sigmoid mapping from expectation score to probability is a convenient score transform and is not guaranteed to be calibrated. No quantum advantage is claimed.

5. Experiment design and evaluation

Once the dataset is downloaded and audited, the CLI accepts an explicit target column, feature columns and optional positive class. It rejects targets with other than two observed classes rather than silently binarizing them. The same holdout rows are used for the classical models and the quantum classifier.

The report records accuracy, balanced accuracy, precision, recall, F1, ROC-AUC when calculable, confusion matrix, per-class report, training time, class counts and input-file SHA-256. The included split is a stratified random holdout baseline. For repeated plots or temporal observations, it must be replaced with a suitable group-aware or time-ordered evaluation.

6. Results and verification status

Dataset-specific benchmark results: not reported in this submission. The actual Kenyan dataset files have not been audited within this package, and no trustworthy experiment report based on those files is available here. Consequently, no dataset row count, accuracy, F1, ROC-AUC, cross-validation score or claimed number of passing tests is asserted as a completed result.

The repository code includes smoke tests for the audit utility, a small quantum-model fit and rejection of unsupported multiclass targets. These tests are intended to check software plumbing; they do not demonstrate agronomic validity, model generalization or quantum advantage. Run the tests in the installed environment and attach the resulting logs and the actual dataset audit before making stronger claims.

7. Reproduction instructions

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"

# Place the official dataset files in data/raw/
python scripts/audit_dataset.py --input data/raw \
  --output data/reports/kenyan_maize_audit.json

# After reviewing the real schema and justifying target/features:
python scripts/train.py --input data/raw/your_table.csv \
  --target YOUR_BINARY_TARGET \
  --features FEATURE_A FEATURE_B FEATURE_C \
  --output-dir models/experiment_001 \
  --report data/reports/experiment_001.json

pytest
streamlit run app/dashboard.py
uvicorn soilq.api:app --reload
```

8. Limitations and responsible use

The prototype is not a farm-specific irrigation recommendation service and does not control pumps or other hardware. It lacks on-farm calibration, external validation and a verified decision label. Statevector simulation is classical computation and can become expensive as qubit count grows. A weaker quantum-model score than a classical baseline is a valid outcome and should be reported honestly.

9. AI tools disclosure

AI assistance was used to help draft and structure project code and documentation. The participant is responsible for reviewing, testing, understanding and correcting the implementation before submission. Any additional tools or assistants used during development should be disclosed accurately according to the hackathon rules.

10. References

1. Kenyan maize field experiment data, Mendeley Data, version 2: https://data.mendeley.com/datasets/8bx88zpvdp/2.

2. Qiskit documentation: https://docs.quantum.ibm.com/.

3. SoilQ repository: https://github.com/Edwin420s/SoilQ.

Prepared for Qiskit Fall Fest Nairobi 2026, Challenge 08 — Quantum AI Classifier.